



## Atividade A5: Mockito na Camada de Serviço, TDD e Trabalho em Equipe

**Objetivo:** Consolidar o uso de Mocks para isolar testes na camada de Serviço (@Service), utilizar o ciclo TDD (Red-Green-Refactor) para projetar regras de negócio e praticar o desenvolvimento colaborativo.

**Valor:** 20 pontos **Dinâmica:** Equipes de até 4 pessoas (com etapa de validação individual).



### Regras de Entrega (Atenção!)

Esta atividade possui uma entrega mista (Individual e/ou em Equipe):

1. **Entrega Individual (Parte 1):** Cada membro da equipe deve fazer o upload de um arquivo avulso chamado `VeterinarioServiceTest_[SeuNome].java` diretamente na plataforma da disciplina (Moodle/Classroom/Teams).
2. **Entrega da Equipe (Parte 2):** A equipe(ou individual) deverá enviar **apenas um link do repositório no GitHub** (ou um arquivo .zip único) contendo o projeto completo com todas as classes de domínio desenvolvidas pelo grupo. OBRIGATÓRIO colocar o nome do responsável nas classes desenvolvidas.

Obs.: Caso faça o trabalho individualmente entregue apenas o link do seu github, com a classe `VeterinarioServiceTest` e a outra service que escolheu.



### Parte 1: Dominando o Mockito no VeterinarioService (INDIVIDUAL)

Nesta etapa, você **não** usará o banco de dados H2. Seu objetivo é testar o comportamento da classe `VeterinarioService` isolando-a com as anotações do Mockito (@Mock no repositório e @InjectMocks no serviço).

*Crie o seu arquivo de teste, resolva os desafios abaixo e envie o arquivo .java com o seu nome na plataforma.*

#### Desafio 1: Testando buscas com Listas

- **Requisito:** Testar o método `buscaVeterinariosComParteNome(String nome)`.
- **Ação:** Crie um cenário no teste onde o mock do repositório é treinado (when) para retornar uma lista com 2 veterinários (criados em memória) quando procurarem pela String "Silva". Execute o método do Service e utilize o `assertEquals` para garantir que a lista retornada tem exatamente o tamanho 2. Não esqueça de usar o `verify()` para checar se o repositório foi chamado!

#### Desafio 2: Protegendo a Exclusão (Tratamento de Exceções)

- **Requisito:** O sistema não pode tentar apagar um veterinário se ele não existir no banco.

- **Ação (Aplique TDD no Service):** 1. **RED:** Escreva um teste chamado `deveLancarExcecaoAoApagarQuandoIdNaoExistir`. Treine o Mock para que a busca pelo ID retorne vazio (`Optional.empty()`). Envolve a chamada de exclusão do service em um `assertThrows` esperando uma `RuntimeException`. 2. **GREEN:** Altere o código real do `VeterinarioService` para fazer o teste passar (buscando o ID primeiro e lançando a exceção se não achar). 3. **ASSERT:** Garanta com o `verify(repository, never()).delete(any())` que o Mockito bloqueou o acesso indevido à exclusão no banco.
- 

## Parte 2: TDD com Mockito (EM EQUIPE)

O sistema da clínica vai expandir e virar um ERP completo! A equipe de Banco de Dados já prometeu entregar as Entidades e os Repositórios prontos. A missão do seu grupo é construir as **Classes de Serviço (@Service)** do zero, guiadas por testes unitários e utilizando Mockito.

**A Dinâmica da Equipe:** Cada membro da equipe deverá escolher **um domínio diferente** da lista abaixo. Vocês trabalharão no mesmo projeto/repositório, mas cada aluno será o "Dono do Produto" do seu respectivo Serviço.

- 🐾 **Aluno 1 (Animal):** (id, nome, especie, idade, internado)
- 📦 **Aluno 2 (Produto):** (id, descricao, preco, quantidadeEstoque, ativo)
- 👤 **Aluno 3 (Funcionario):** (id, nome, cargo, salario, emFerias)
- 🛠️ **Aluno 4 (Servico):** (id, nome, valor, tempoMinutos, disponivel)

(Nota: Mantenham as entidades simples, **sem relacionamentos** com outras tabelas por enquanto).

**Preparação do Cenário (Para cada domínio):** Para o código compilar, criem o básico:

1. A classe da Entidade com seus atributos básicos, construtores e getters/setters.
2. Uma interface vazia `SeuDominioRepository` (ex: `ProdutoRepository`).

**Regra de Ouro:** Você **NÃO** pode criar nenhum método na classe de Serviço antes de escrever o teste falhando (RED) que obrigue a existência dele!

### Ciclo 1: Regra de Negócio no Cadastro (Status Padrão)

- **Requisito:** Todo registro que entra no sistema pelo método de cadastro deve receber um status padrão antes de ser salvo (ex: um `Animal` entra como `internado = true`; um `Produto` entra como `ativo = true`).
- **RED:** Na sua classe de teste (`SeuDominioServiceTest`), instancie o seu objeto com o status oposto (ex: `ativo = false`). Ensine o mock do repositório a retornar o objeto salvo. Chame o método de cadastrar no Service (use a IDE para criar o método que ainda não existe).
- **GREEN:** Implemente o método no Service, forçando a mudança do atributo para o valor correto e retornando o `repository.save()`.

- **Asserts:** Verifique se o objeto retornado está com o status correto e use o `verify` para garantir que o mock do repositório salvou os dados.

## Ciclo 2: Proteção de Domínio (Validação)

- **Requisito:** O sistema deve barrar dados inválidos no cadastro. (Ex: Animal não pode ser de espécie não atendida; Produto não pode ter preço negativo; Funcionário não pode ter salário menor que o mínimo).
- **RED:** Crie um teste instanciando um objeto com o dado inválido. Envolve a chamada do método cadastrar em um `assertThrows` esperando uma `IllegalArgumentException`.
- **GREEN:** No método do Service, adicione um `if` para validar o campo e lançar a exceção.
- **Asserts:** Use o `verify(repository, never()).save(any())` para provar que a "outra equipe" não vai receber dados inválidos no banco!

## Ciclo 3: Regra de Negócio de Atualização (Ação Específica)

- **Requisito:** Crie um método com um nome de negócio claro que atualize um status ou valor. (Ex: `darAlta(id)` para Animal; `inativar(id)` para Produto; `concederFerias(id)` para Funcionário).
- **RED:** Crie o teste. Treine o mock do repositório para retornar um objeto válido quando buscarem pelo ID. Chame o seu novo método de atualização.
- **GREEN:** No Service, implemente a lógica: busque o objeto pelo ID (lance erro se não achar), altere o atributo necessário e chame `repository.save()`.
- **Asserts:** Verifique se o `save` foi chamado usando o `verify`.

---

Podem criar novos testes e funcionalidades.